



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR
Membre de
HONORIS UNITED UNIVERSITIES



CYBERLENS

CYBER SECURITY SOLUTIONS

Rapport De Projet:

Plateforme de centralisation et d'analyse des logs de sécurité

Filière : Cybersécurité & Réseaux

Réalisé Par:

Ahmed Houachmi

Rachid Yasser

Encadré Par:

Mr Nadir Younes

Année universitaire 2025/2026

Remerciement

Avant toute chose, nous souhaitons exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de manière directe ou indirecte, à la réalisation de ce projet.

Nous remercions tout particulièrement **Mr Nadir Younes** pour sa disponibilité, sa rigueur et ses précieux conseils tout au long de ce travail. Son encadrement a été essentiel pour structurer nos idées et orienter nos efforts dans la bonne direction.

Nos remerciements vont aussi à **nos camarades**, avec qui nous avons partagé des moments d'échange, de réflexion et de collaboration enrichissants.

Enfin, nous remercions toute personne ayant, à un moment ou un autre, apporté une aide, un avis ou un encouragement, aussi minime soit-il. Chaque geste a compté dans la progression et l'aboutissement de ce travail.

Résumé

Ce projet s'inscrit dans le cadre de la mise en place d'une plateforme pédagogique de centralisation et d'analyse des logs de sécurité. L'objectif principal est de collecter des journaux provenant de différentes sources, de les normaliser dans un format commun, puis de permettre leur exploitation à travers des fonctionnalités de recherche, de filtrage et de visualisation.

La plateforme développée qui est nommée **CyberLens**, vise à faciliter l'investigation des événements de sécurité en offrant une vue unifiée des activités du système. Elle permet notamment d'identifier des comportements suspects tels que les tentatives de connexion échouées, les accès non autorisés ou encore les communications réseau anormales.

Dans une approche simplifiée et adaptée à un contexte académique, ce projet met l'accent sur la compréhension des mécanismes fondamentaux de gestion des logs, sans prétendre reproduire un système **SIEM** industriel. Il constitue néanmoins une base solide pour appréhender les enjeux liés à la supervision et à la détection des incidents de sécurité.

Abstract

This project is part of the development of an educational platform for centralizing and analyzing security logs. The main objective is to collect logs from various sources, standardize them into a common format, and then enable their use through search, filtering, and visualization functionalities.

The platform developed, named **CyberLens**, aims to facilitate the investigation of security events by providing a unified view of system activity. It allows, in particular, the identification of suspicious behavior such as failed login attempts, unauthorized access, and abnormal network communications.

Using a simplified approach adapted to an academic context, this project emphasizes understanding the fundamental mechanisms of log management, without attempting to replicate an industrial SIEM system. Nevertheless, it provides a solid foundation for understanding the challenges related to monitoring and detecting security incidents.

Sommaire

Remerciements	2
Résumé	3
Abstract	4
Sommaire	5
Liste d'abreviations	8
Mots clés	9
Liste des tableaux	10
 Introduction Générale	11
1. Contexte général du projet	11
2. Problématique traitée	11
3. Objectifs généraux et spécifiques	11
4. Méthodologie globale adoptée	12
5. Principaux résultats attendus	12
6. Organisation du rapport	12
 Chapitre 1 : Présentation générale du projet	13
1.1 Présentation du domaine d'application	14
1.2 Besoin identifié	14
1.3 Problématique détaillée	14
1.4 Périmètre fonctionnel	14
1.5 Périmètre technique	15
1.6 Technologies utilisées	15
1.7 Limites et hypothèses du projet	16
 Chapitre 2 : État de l'art de l'existant	17
2.1 Types de logs de sécurité	18
2.2 Centralisation et normalisation	18
2.3 SIEM open-source et propriétaires	18
2.4 Recherche et corrélation d'événements	18
2.5 Problèmes de volumétrie et de bruit	18
2.6 Comparaison avec les solutions existantes	19
2.7 Positionnement de la solution proposée	19
 Chapitre 3 : Analyse du cahier des charges	20
3.1 Identification des acteurs	21
3.2 Besoins fonctionnels	21

3.3 Besoins non fonctionnels	21
3.4 Contraintes techniques et matérielles	21
3.5 Contraintes de sécurité	22
3.6 Critères d'acceptation	22
Chapitre 4 : Gestion de projet	23
4.1 Méthodologie de gestion adoptée	24
4.2 Découpage du projet en lots de travail	24
4.3 Répartition des tâches	24
4.4 Gestion des risques	25
4.5 Planning prévisionnel et diagramme de Gantt	25
4.6 Suivi de l'avancement	26
4.7 Difficultés rencontrées et solutions appliquées	26
4.8 Bilan de la collaboration	26
Chapitre 5 : Conception de la solution	27
5.1 Architecture globale de CyberLens	28
5.2 Sources de logs	28
5.3 Pipeline d'ingestion	28
5.4 Agent ou forwarder de collecte	28
5.5 Stockage centralisé	29
5.6 Moteur d'analyse	30
5.7 Tableaux de bord et alertes	30
5.8 Diagramme de cas d'utilisation	31
5.9 Diagramme de séquence / flux	32
5.10 Diagramme de déploiement	33
5.11 Description détaillée des modules	34
Chapitre 6 : Réalisation et implémentation	35
6.1 Préparation de l'environnement	36
6.2 Configuration des sources de logs	36
6.3 Déploiement de la plateforme	36
6.4 Création des index	37
6.5 Configuration des recherches	38
6.6 Création des tableaux de bord	39
6.7 Structure du projet et organisation des fichiers	39
6.8 Captures d'écran commentées	40
6.9 Difficultés techniques rencontrées	44
Chapitre 7 : Sécurité, conformité et aspects éthiques	45
7.1 Protection des logs sensibles	46

7.2 Contrôle d'accès au tableau de bord	46
7.3 Conservation et intégrité des journaux	46
7.4 Traçabilité des actions administrateur	46
7.5 Cadre légal d'utilisation	46
7.6 Bonnes pratiques recommandées	47
Chapitre 8 : Tests, validation et résultats	48
8.1 Tests de collecte	49
8.2 Tests d'indexation	49
8.3 Tests de recherche	49
8.4 Simulation d'événements suspects	50
8.5 Validation d'alertes	50
8.6 Mesure du délai de remontée	51
8.7 Stratégie de test	51
8.8 Tableau de validation fonctionnelle	52
8.9 Analyse des résultats obtenus	53
8.10 Comparaison avec les objectifs initiaux	53
Chapitre 9 : Dimension entrepreneuriale et valorisation	54
9.1 Besoin marché identifié	55
9.2 Utilisateurs ou clients cibles	55
9.3 Proposition de valeur	55
9.4 Analyse de la concurrence	55
9.5 Business Model Canvas	56
9.6 Modèle économique possible	57
9.7 Stratégie de déploiement ou de commercialisation	57
9.8 Solution de supervision sécurité pour PME	57
9.9 Service d'installation SIEM	57
9.10 Offre de tableaux de bord prêts à l'emploi	58
9.11 Pack de formation à l'analyse de logs	58
Chapitre 10 : Bilan général et perspectives	59
10.1 Bilan technique du projet	60
10.2 Bilan fonctionnel	60
10.3 Compétences acquises	60
10.4 Limites de la solution	60
10.5 Améliorations futures	61
10.6 Perspectives d'industrialisation	61
Conclusion Générale	62
Bibliographie et Webographie	64

Liste d'abbreviations

SIEM	Security Information and Event Management
SOC	Security Operations Center
SSH	Secure Shell
UFW	Uncomplicated Firewall
SQL	Structured Query Language
XSS	Cross-Site Scripting
LFI	Local File Inclusion
HTTP	Hypertext Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
API	Application Programming Interface
IP	Internet Protocol
KPI	Key Performance Indicator
JSON	JavaScript Object Notation
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
AJAX	Asynchronous JavaScript and XML
WAL	Write-Ahead Logging (mode SQLite)
DVWA	Damn Vulnerable Web Application
BMC	Business Model Canvas
PME	Petite et Moyenne Entreprise
SaaS	Software as a Service

Mots-clés et keywords

Mots-clés

Logs	Centralisation
SIEM	Indexation
Alertes	Tableaux de bord
Cybersécurité	Python
Flask	SQLite

Keywords

Logs	Centralization
SIEM	Indexing
Alerts	Dashboards
Cybersecurity	Python
Flask	SQLite

Liste des tableaux

Tab. 1.1	Technologies utilisées dans le projet
Tab. 2.1	Comparaison avec les solutions SIEM existantes
Tab. 3.1	Matrice de traçabilité des besoins
Tab. 4.1	Répartition des tâches entre les membres du groupe
Tab. 4.2	Planning prévisionnel (diagramme de Gantt)
Tab. 4.3	Gestion des risques du projet
Tab. 5.1	Schéma de la table logs
Tab. 5.2	Niveaux de sévérité des événements
Tab. 5.3	Routes de l'application Flask
Tab. 5.4	Description détaillée des modules
Tab. 6.1	Index créés sur la base de données
Tab. 6.2	Requêtes SQL des mécanismes de recherche
Tab. 8.1	Résultats des tests de recherche
Tab. 8.2	Validation des alertes par scénario d'attaque
Tab. 8.3	Mesure du délai de remontée
Tab. 8.4	Tableau de validation fonctionnelle
Tab. 8.5	Comparaison avec les objectifs initiaux

Introduction Générale

1. CONTEXTE GENERAL DU PROJET

Avec l'augmentation constante des menaces informatiques et la complexité croissante des systèmes d'information, la gestion et l'analyse des logs de sécurité sont devenues des éléments essentiels pour assurer la protection des infrastructures numériques. Les journaux générés par les systèmes, les applications et les équipements réseau constituent une source précieuse d'informations permettant de détecter, analyser et comprendre les incidents de sécurité.

2. PROBLEMATIQUE TRAITEE

Les logs sont souvent hétérogènes, volumineux et difficiles à exploiter sans outils adaptés. Dispersés sur plusieurs machines et présentés dans des formats différents, leur analyse est complexe et chronophage. L'absence d'un système centralisé complique la détection rapide des anomalies et retarde la réponse aux incidents de sécurité.

3. OBJECTIFS GENERAUX ET SPECIFIQUES

L'objectif principal est de concevoir et développer une plateforme permettant la centralisation, le stockage et l'analyse des logs de sécurité. Plus spécifiquement, le projet vise à :

- Collecter des logs provenant de différentes sources (auth.log, access.log, ufw.log)
- Normaliser ces logs afin de les rendre exploitables dans un format unique
- Stocker les événements dans une base de données structurée
- Détecter automatiquement les comportements suspects par une logique de classification
- Permettre la recherche, le filtrage et la visualisation des événements en temps réel

4. METHODOLOGIE GLOBALE ADOPTEE

Le projet adopte une approche itérative et incrémentale, en développant successivement chaque composant du pipeline : identification des sources, définition du schéma de normalisation, implémentation des scripts d'ingestion, développement du tableau de bord, puis tests et validation.

5. PRINCIPAUX RESULTATS ATTENDUS

À l'issue du projet, la plateforme CyberLens devra être capable de collecter, normaliser et visualiser en temps réel les événements de sécurité issus de trois sources distinctes, tout en détectant automatiquement les comportements suspects selon une taxonomie définie.

6. ORGANISATION DU RAPPORT

Ce rapport est structuré en dix chapitres : après une présentation générale du projet et un état de l'art, le cahier des charges est analysé en détail. Suivent la gestion de projet, la conception de la solution, son implémentation, les aspects sécurité et conformité, les tests et résultats, la dimension entrepreneuriale et le bilan général.

CHAPITRE 1 :

Présentation

générale du projet

1.1 PRESENTATION DU DOMAINE D'APPLICATION

Ce projet s'inscrit dans le domaine de la cybersécurité défensive, et plus précisément dans la gestion des événements et informations de sécurité (SIEM). Face à la prolifération des cybermenaces, les organisations ont besoin d'outils permettant de surveiller en continu l'activité de leurs systèmes et de détecter rapidement les comportements anormaux.

1.2 BESOIN IDENTIFIE

Dans un contexte académique, il n'existe pas de plateforme de supervision simple et pédagogique permettant de comprendre le fonctionnement d'un SIEM. Les solutions industrielles (Splunk, Elastic SIEM) sont complexes à déployer et à comprendre. Il y a donc un besoin réel d'une solution légère, documentée et explicite, démontrant les principes fondamentaux.

1.3 PROBLEMATIQUE DETAILLEE

Les systèmes d'information génèrent en permanence une grande quantité de journaux d'événements. Ces logs sont essentiels pour détecter les incidents de sécurité, analyser les comportements suspects et assurer la traçabilité des actions. Cependant, leur hétérogénéité et leur volume en rendent l'exploitation manuelle impossible sans outillage adapté.

1.4 PERIMETRE FONCTIONNEL

Le périmètre fonctionnel du projet couvre les fonctionnalités suivantes :

- Collecte de logs depuis trois sources : authentification, accès web, pare-feu
- Normalisation et classification automatique des événements
- Stockage structuré dans une base de données relationnelle
- Détection de comportements suspects (brute force, injection SQL, XSS, LFI...)
- Visualisation interactive via un tableau de bord web
- Filtrage dynamique par source, sévérité et recherche textuelle

1.5 PERIMETRE TECHNIQUE

La solution repose sur un environnement Linux (Ubuntu 24), Python 3.12 pour les scripts d'ingestion et le backend Flask, SQLite comme base de données embarquée, et Chart.js pour la visualisation. Kali Linux est utilisé pour simuler des attaques lors des tests.

1.6 Technologies utilisées :

Technologie	Role	version
Python 3.12	Scripts d'ingestion et backend	3.12.3
Flask	Framework web (API + rendu HTML)	Latest
SQLite	Base de données relationnelle légère	Embarquée
Chart.js	Visualisation des graphiques	—
HTML/CSS/JS	Interface du tableau de bord	—
Ubuntu 24 (linux)	Environnement d'exécution	24 LTS
Kali Linux	Simulateur d'attaque	Rolling

1.7 LIMITES ET HYPOTHESES DU PROJET :

Ce projet adopte une approche pédagogique et ne prétend pas reproduire un SIEM industriel. Les limites principales sont : utilisation de SQLite (non adapté à la production), absence d'authentification avancée, sources de logs limitées à trois types, et environnement mono-machine simulé.

CHAPITRE 2 :état

de l'art de

l'existant

2.1 TYPES DE LOGS DE SECURITE

Les logs de sécurité se classifient en plusieurs catégories selon leur source : logs système (auth, syslog), logs applicatifs (Apache, Nginx), logs réseau (pare-feu, IDS), et logs de base de données. Chaque type présente un format spécifique et des informations caractéristiques nécessitant des parseurs dédiés.

2.2 CENTRALISATION ET NORMALISATION

La centralisation des logs consiste à regrouper tous les journaux en un point unique de collecte, facilitant leur corrélation et leur analyse. La normalisation vise à transformer des logs hétérogènes en un format homogène, permettant d'appliquer des règles de détection uniformes indépendamment de la source.

2.3 SIEM OPEN-SOURCE ET PROPRIETAIRES

Plusieurs solutions SIEM existent sur le marché. Parmi les solutions open-source, on trouve ELK Stack (Elasticsearch, Logstash, Kibana), Wazuh et OSSEC. Les solutions propriétaires incluent Splunk, IBM QRadar et Microsoft Sentinel. Ces outils offrent des fonctionnalités avancées mais nécessitent des ressources importantes pour leur déploiement.

2.4 RECHERCHE ET CORRELATION D'EVENEMENTS

La corrélation d'événements consiste à identifier des relations entre des événements distincts pour détecter des attaques complexes. Par exemple, plusieurs échecs d'authentification sur une même IP sur une courte période peuvent signaler une attaque par brute force.

2.5 PROBLEMES DE VOLUMETRIE ET DE BRUIT

Un système SIEM en production peut traiter des millions d'événements par jour. Le défi principal est de filtrer le bruit (faux positifs) pour ne conserver que les événements significatifs. Les techniques de déduplication, de seuillage et de pondération sont essentielles pour maintenir la pertinence des alertes.

2.6 COMPARAISON AVEC LES SOLUTIONS EXISTANTES

Solution	Open-Source	Complexité	Ressources	Cas d'usage
ELK Stack	Oui	Haute	Élevées	Production
Wazuh	Oui	Moyenne	Moyennes	PME/SOC
Splunk	Non	Haute	Très élevées	Enterprise
CyberLens	Oui	Faible	Très faibles	Académique

2.7 POSITIONNEMENT DE LA SOLUTION PROPOSÉE

CyberLens se positionne comme une solution légère et pédagogique, conçue pour comprendre et démontrer les mécanismes d'un SIEM sans la complexité des solutions industrielles. Elle constitue un excellent point d'entrée pour les étudiants et professionnels souhaitant maîtriser les concepts de base avant de migrer vers des solutions plus robustes.

CHAPITRE 3 :

Analyse de cahier

de charges

3.1 IDENTIFICATION DES ACTEURS

Le système CyberLens interagit avec deux catégories d'acteurs principaux :

- Analyste SOC / Administrateur : consulte le tableau de bord, applique des filtres, analyse les événements de sécurité
- Système (automatique) : les scripts d'ingestion alimentent la base de données de manière autonome en temps réel

3.2 BESOINS FONCTIONNELS

- BF01 — Collecte automatique et continue des logs depuis auth.log, access.log et ufw.log
- BF02 — Normalisation des logs dans un format unifié (timestamp, source, type, sévérité, IP, message)
- BF03 — Détection automatique de 13 types d'événements suspects
- BF04 — Corrélation pour la détection du brute force SSH (seuil : 5 échecs par IP)
- BF05 — Tableau de bord interactif avec KPIs, graphiques et tableau des événements
- BF06 — Filtrage dynamique par source, sévérité et recherche textuelle
- BF07 — Rafraîchissement automatique toutes les 2 secondes

3.3 BESOINS NON FONCTIONNELS

- BNF01 — Performance : traitement des logs en moins de 2 secondes par cycle
- BNF02 — Fiabilité : éviter les doublons dans la base de données
- BNF03 — Maintenabilité : architecture modulaire, code documenté
- BNF04 — Portabilité : fonctionnement sur tout système Linux avec Python 3.x
- BNF05 — Extensibilité : facilité d'ajout de nouvelles sources de logs

3.4 CONTRAINTES TECHNIQUES ET MATERIELLES

La solution doit fonctionner sur un ordinateur standard avec Ubuntu 24 LTS. Elle doit utiliser exclusivement des technologies open-source. La base de données doit être

embarquée (pas de serveur dédié requis). L'interface web doit être accessible depuis un navigateur moderne sans installation supplémentaire.

3.5 CONTRAINTES DE SECURITE

Les logs traités peuvent contenir des données sensibles (adresses IP, noms d'utilisateurs). L'accès au tableau de bord doit être restreint au réseau local. Les logs ne doivent pas être modifiés lors de la lecture. La plateforme elle-même doit être sécurisée contre les injections.

3.6 CRITERES D'ACCEPTATION

La solution sera considérée comme valide si : la collecte fonctionne en continu sans interruption, au moins 10 types d'événements sont détectés correctement, le tableau de bord se rafraîchit automatiquement, les filtres fonctionnent correctement, et les tests de simulation d'attaques génèrent les alertes attendues.

CHAPITRE 4 :

Gestion de projet

4.1 METHODOLOGIE DE GESTION ADOPTEE

Le projet a été conduit selon une approche itérative et incrémentale, inspirée des méthodes Agile. Chaque itération correspond à un composant fonctionnel du système : identification des sources, schéma de données, pipeline d'ingestion, tableau de bord, tests. Cette approche permet d'identifier rapidement les problèmes et d'ajuster le développement en conséquence.

4.2 DECOUPAGE DU PROJET EN LOTS DE TRAVAIL

- Lot 1 : Étude de l'existant et définition des spécifications (Semaine 1-2)
- Lot 2 : Identification des sources et définition du schéma de normalisation (Semaine 3)
- Lot 3 : Implémentation du pipeline d'ingestion (ingest_auth, ingest_apache, ingest_firewall) (Semaine 4-5)
- Lot 4 : Développement de l'application Flask et du tableau de bord (Semaine 6-7)
- Lot 5 : Tests, validation et correction des bugs (Semaine 8)
- Lot 6 : Rédaction du rapport et préparation de la soutenance (Semaine 9-10)

4.3 RÉPARTITION DES TACHES

Tâche	Ahmed Houachmi	Rachid Yasser
Pipeline d'ingestion	Principal	Support
Base de données SQLite	Principal	Support
Application Flask	Support	Principal
Interface tableau de bord	Support	Principal
Tests et validation	Conjoint	Conjoint
Rédaction du rapport	Conjoint	Conjoint

4.4 GESTION DES RISQUES

Risque identifié	Probabilité	Mesure de mitigation
Format de log inattendu	Moyenne	Utilisation d'expressions régulières robustes avec gestion d'exceptions
Performance insuffisante de SQLite	Faible	Indexation des colonnes fréquemment interrogées
Conflits de dépendances Python	Faible	Utilisation d'un environnement virtuel dédié
Faux positifs dans la détection	Haute	Affinement des règles de classification par tests répétés

4.5 PLANNING PREVISIONNEL ET DE DIAGRAMME DE GANTT:

DIAGRAMME DE GANTT – PLANIFICATION DU PROJET

LOT	DESCRIPTION	SEMAINES									
		1	2	3	4	5	6	7	8	9	10
LOT 1	Étude de l'existant et définition des spécifications										
LOT 2	Identification des sources et définition du schéma de normalisation										
LOT 3	Implémentation du pipeline d'ingestion (ingest_auth, ingest_apache, ingest_firewall)										
LOT 4	Développement de l'application Flask et du tableau de bord										
LOT 5	Tests, validation et correction des bugs										
LOT 6	Rédaction du rapport et préparation de la soutenance										

4.6 SUIVI DE L'AVANCEMENT

L'avancement du projet a été suivi au moyen de réunions hebdomadaires courtes entre les deux membres du binôme, permettant de comparer le travail réalisé au planning prévisionnel et de réajuster la charge si nécessaire. Un tableau de suivi simple (sous forme de checklist partagée) a été utilisé pour cocher les tâches terminées, en cours ou bloquées. Les écarts les plus significatifs sont survenus lors du Lot 3, où le temps consacré au parsing des formats de logs a été sous-estimé d'environ deux jours, compensé par une accélération du Lot 4 grâce à la réutilisation de composants HTML/CSS communs.

4.7 DIFFICULTES RENCONTREES ET SOLUTIONS APPLIQUEES

- Hétérogénéité des formats de logs : résolue par la création d'expressions régulières spécifiques à chaque source
- Gestion des doublons Apache : résolue par une vérification SELECT avant chaque INSERT
- Performance du rafraîchissement temps réel : optimisée par des requêtes SQL indexées et un appel AJAX asynchrone
- Sous-estimation initiale du temps de parsing : compensée par une réorganisation du planning entre les Lots 3 et 4

4.8 BILAN DE LA COLLABORATION ET ORGANISATION DU TRAVAIL

La collaboration au sein du binôme s'est appuyée sur une répartition claire des responsabilités techniques (ingestion et base de données d'un côté, application Flask et interface de l'autre), tout en gardant des moments de travail conjoint pour les tests et la rédaction. L'utilisation d'un dépôt Git partagé a permis de suivre l'évolution du code et d'éviter les conflits de version. Cette organisation a favorisé une bonne complémentarité des compétences et un respect global du planning prévisionnel, avec un seul ajustement notable au niveau du Lot 3.

CHAPITRE 5:

Conception de la

solution

5.1 ARCHITECTURE GLOBALE DE CYBERLENS

CyberLens repose sur une architecture en pipeline à trois niveaux : la couche de collecte (scripts d'ingestion), la couche de stockage (SQLite), et la couche de présentation (Flask + Chart.js). Un moniteur temps réel orchestre l'ensemble en boucle continue.

5.2 SOURCES DE LOGS

Trois sources de logs ont été sélectionnées pour couvrir les principaux vecteurs d'attaque :

- `auth.log` : journaux d'authentification SSH Linux — détection de brute force, connexions suspectes
- `access.log` : journaux d'accès Apache — détection d'injections SQL, XSS, LFI, scans
- `ufw.log` : journaux du pare-feu UFW — détection de blocages et connexions anormales

5.3 PIPELINE D'INGESTION

Le pipeline est composé de quatre composants principaux :

- `ingest_auth.py` : parse `auth.log`, détecte les échecs SSH et le brute force (seuil 5 échecs)
- `ingest_apache.py` : parse `access.log` avec regex, détecte 7 types d'attaques web
- `ingest_firewall.py` : parse `ufw.log`, classe BLOCK vs événements normaux
- `realtime_monitor.py` : orchestre l'ingestion en boucle de 2 secondes

5.4 AGENT OU FORWARDER DE COLLECTE

Plutôt qu'un agent dédié de type Filebeat ou Fluentd, le choix a été fait d'implémenter un forwarder léger en Python : `realtime_monitor.py`. Ce choix réduit la complexité de déploiement (aucun service tiers à installer) tout en conservant le principe d'un forwarder classique : lecture incrémentale des fichiers source via un pointeur de position (`.pos`), détection de nouvelles lignes, et transmission aux scripts d'ingestion appropriés selon la source.

5.4 STOCKAGE CENTRALISE

Le stockage repose sur une base de données SQLite unique (cyberlens.db), choisie pour sa simplicité de déploiement (aucun serveur de base de données à administrer) et son adéquation avec le volume de données attendu dans un contexte académique. La table principale 'logs' centralise tous les événements normalisés :

Champ	Type	Description
timestamp	DATETIME	Horodatage de l'événement
source	TEXT	Source du log (apache / auth.log / firewall)
event_type	TEXT	Type d'événement détecté
severity	TEXT	Niveau de criticité (low/medium/high/critical)
src_ip	TEXT	Adresse IP source (nullable)
message	TEXT	Message brut ou description de l'événement

5.5 MOTEUR D'ANALYSE

Le moteur d'analyse applique des règles de classification à chaque ligne ingérée, en s'appuyant sur des expressions régulières et des compteurs en mémoire pour la corrélation. Il attribue à chaque événement un type (13 types détectés) et un niveau de sévérité :

Niveau	Couleur	Description	Exemples
LOW	Vert	Activité normale	Connexion réussie, requête web standard
MEDIUM	Jaune	Activité potentiellement suspecte	Scan réseau, tentative de login
HIGH	Orange	Activité suspecte ou anormale	Échec d'authentification, XSS, blocage pare-feu
CRITICAL	Rouge	Attaque confirmée	Injection SQL, brute force SSH détecté

5.6 TABLEAUX DE BORD ET ALERTES

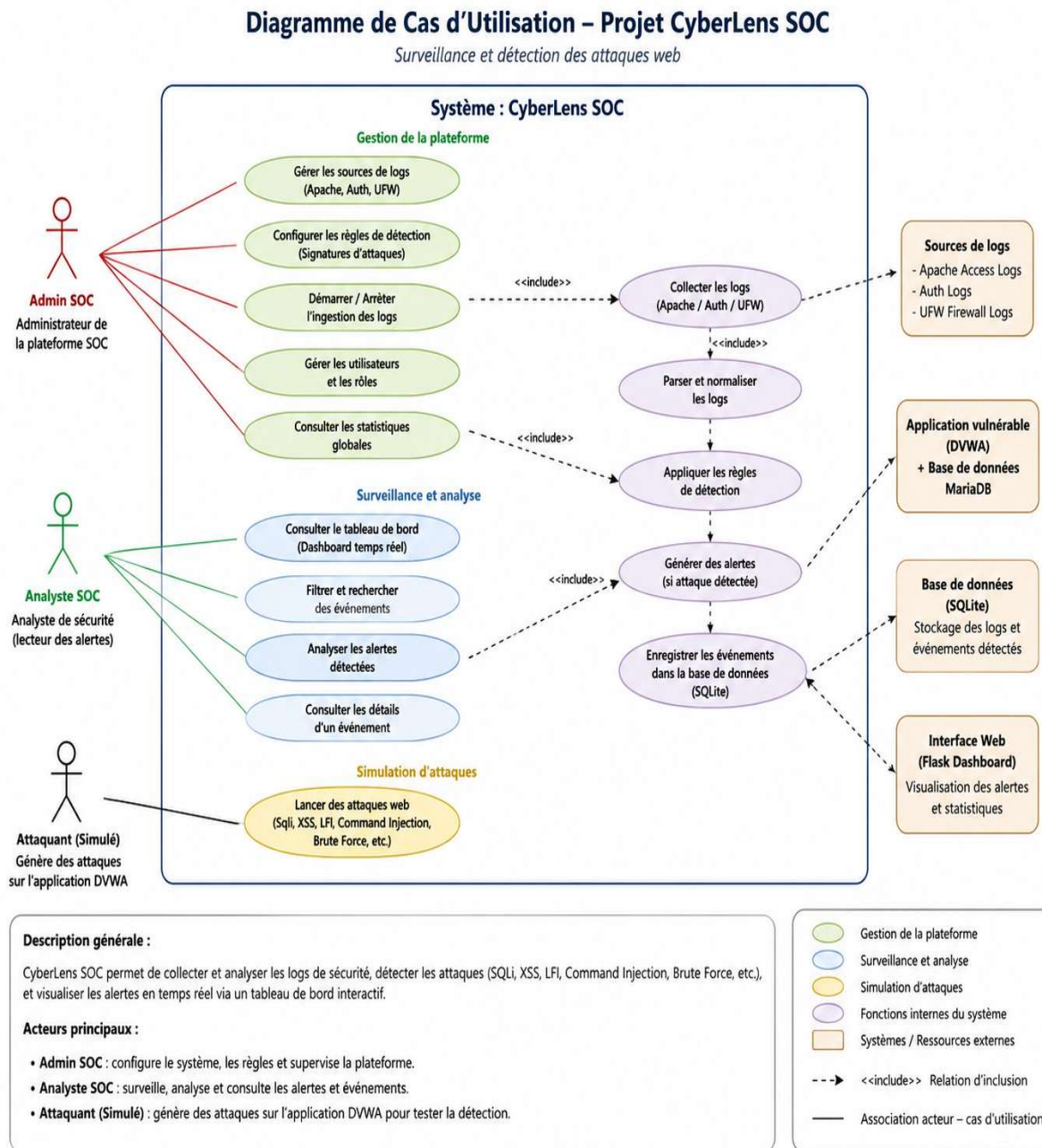
L'application Flask expose deux routes et propose un tableau de bord interactif :

- 3 indicateurs KPI : Total Logs, Total Attaques, Top Attaquant (IP la plus active)
- Graphique temporel (line chart) : événements sur les 15 dernières minutes
- Graphique camembert (pie chart) : répartition par type d'attaque
- Graphique anneau (doughnut) : répartition par niveau de sévérité
- Tableau des 100 derniers événements, colorés selon la sévérité

Les alertes ne sont pas envoyées par un canal externe (email/SMS) dans cette version : l'alerte se matérialise par l'apparition immédiate de l'événement CRITICAL en rouge dans le tableau, visible dès le rafraîchissement suivant (moins de 4 secondes après l'écriture du log).

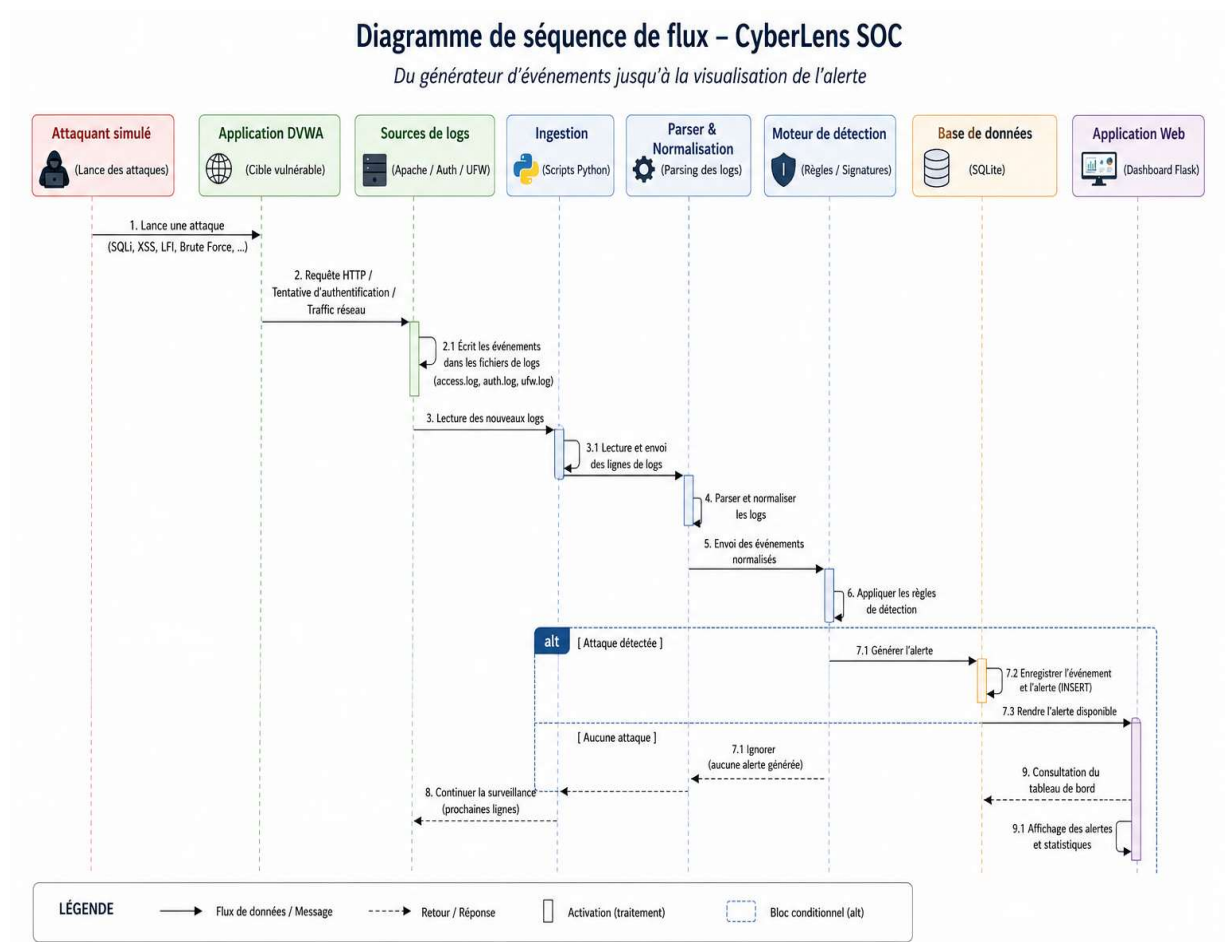
5.7 DIAGRAMME DE CAS D'UTILISATION

Le diagramme ci-dessous résume les interactions entre les deux acteurs du système et les fonctionnalités principales



Ce diagramme de cas d'utilisation présente le fonctionnement du système **CyberLens SOC** pour la surveillance et la détection des attaques web. L'**Admin SOC** configure la plateforme, les sources de logs et les règles de détection. Les logs sont collectés, analysés et stockés afin de générer des alertes en cas d'attaque. L'**Analyste SOC** consulte le tableau de bord pour surveiller et analyser les événements détectés. Un **attaquant simulé** lance des attaques sur l'application DVWA pour tester l'efficacité du système.

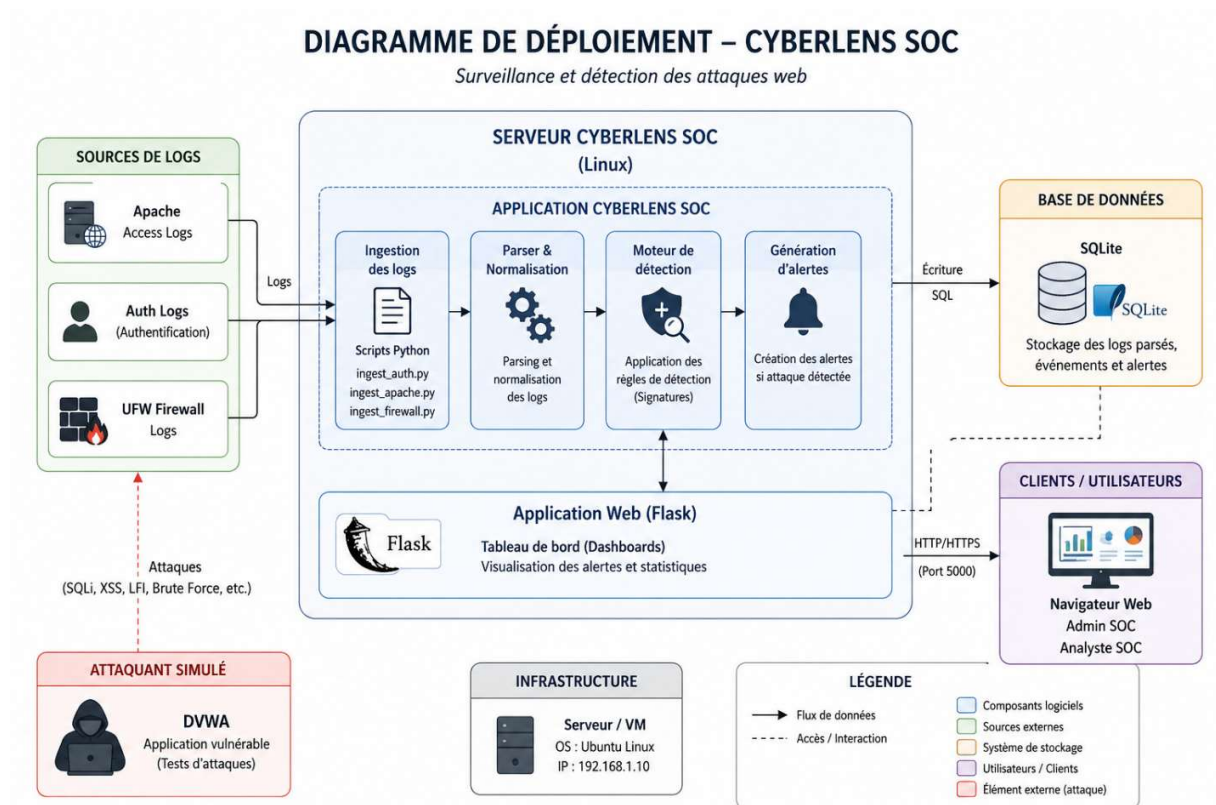
5.8 DIAGRAMME DE SÉQUENCE / FLUX



Ce diagramme de séquence de flux montre le parcours des données dans **CyberLens SOC**, depuis le lancement d'une attaque jusqu'à l'affichage de l'alerte. Un attaquant simulé envoie des requêtes vers l'application DVWA, ce qui génère des logs (Apache, Auth, UFW). Ces logs sont collectés par les scripts

d'ingestion, puis parsés et normalisés avant d'être analysés par le moteur de détection. Si une attaque est identifiée, une alerte est générée, enregistrée dans la base de données et affichée sur le tableau de bord web ; sinon, les événements sont simplement ignorés et la surveillance continue.

5.9 DIAGRAMME DE DÉPLOIEMENT



Le diagramme de déploiement illustre l'architecture de CyberLens SOC. Trois sources de logs (Apache, auth.log, UFW) alimentent le serveur Linux, où un pipeline Python ingère, parse et normalise les données avant qu'un moteur de détection applique des règles de signatures et génère des alertes en cas d'attaque. Une application Flask expose un tableau de bord accessible en HTTP/HTTPS (port 5000), pendant que les logs et alertes sont stockés dans une base SQLite. Les analystes SOC consultent ce tableau de bord via leur navigateur. Un attaquant simulé exploite une application vulnérable (DVWA) pour générer des attaques (SQLi, XSS, LFI, brute force), bouclant ainsi le scénario de test.

5.10 DESCRIPTION DETAILLEE DES MODULES

Récapitulatif des modules logiciels constituant CyberLens, avec leur rôle et leurs entrées/sorties.

Module	Rôle	Entrées / Sorties
ingest_auth.py	Parse auth.log, détecte échecs SSH et brute force	Entrée : auth.log — Sortie : table logs
ingest_apache.py	Parse access.log, détecte 7 types d'attaques web	Entrée : access.log — Sortie : table logs
ingest_firewall.py	Parse ufw.log, classe BLOCK/normal	Entrée : ufw.log — Sortie : table logs
realtime_monitor.py	Orchestre les 3 scripts en boucle de 2s	Entrée : fichiers .pos — Sortie : appels d'ingestion
init_db.py	Crée le schéma SQLite et les index	Sortie : cyberlens.db
app.py (Flask)	Sert le tableau de bord et l'API JSON	Entrée : cyberlens.db — Sortie : HTML / JSON

CHAPITRE 6:

Réalisation et

implémentation

6.1 PREPARATION DE L'ENVIRONNEMENT

L'environnement de développement repose sur Ubuntu 24 LTS avec Python 3.12. Les dépendances sont installées via pip (Flask) et les fichiers de logs sont fournis par le système. Un environnement virtuel Python a été créé pour isoler les dépendances du projet.

6.2 CONFIGURATION DES SOURCES DE LOGS

Apache est configuré pour écrire dans `/var/log/apache2/access.log` au format Combined Log Format. Le service SSH écrit dans `/var/log/auth.log`. UFW (pare-feu) est activé et configuré pour journaliser les connexions bloquées dans `/var/log/ufw.log`.

6.3 DEPLOIEMENT DE LA PLATEFORME

Le déploiement s'effectue en trois étapes : initialisation de la base de données (python `init_db.py`), démarrage du moniteur d'ingestion en arrière-plan (python `realtime_monitor.py`), puis lancement de l'application Flask (python `app.py`), accessible sur le réseau local au port 5000. Les trois processus tournent en parallèle sur la même machine, comme détaillé dans le diagramme de déploiement du Chapitre 5.

PIPELINE D'INGESTION — DETAIL DES SCRIPTS

Le script `ingest_auth.py` analyse `auth.log` ligne par ligne à l'aide d'expressions régulières pour extraire les tentatives de connexion SSH. Il maintient un compteur en mémoire par adresse IP et génère un événement CRITICAL de type `SSH_BRUTE_FORCE` dès que le seuil de 5 échecs est dépassé.

Le script `ingest_apache.py` parse `access.log` avec une regex couvrant le Combined Log Format. Il applique une série de règles de détection : injection SQL, XSS, LFI, injection de commandes, et scans réseau, avec une vérification de doublon avant chaque insertion.

Le script `ingest_firewall.py` analyse `ufw.log` pour extraire les IP source et destination, et classe chaque événement en BLOCK (sévérité HIGH) ou événement normal (sévérité LOW). Un fichier

.pos assure, pour chaque script, la lecture incrémentale et évite la re-lecture des lignes déjà traitées.

6.4 CREATION DES INDEX

Afin de garantir des temps de réponse acceptables malgré le rafraîchissement automatique toutes les 2 secondes, plusieurs index ont été créés sur la table logs lors de l'initialisation de la base de données :

Colonne indexée	Type d'index	Justification
timestamp	B-Tree (CREATE INDEX)	Tri chronologique et fenêtres glissantes (15 dernières minutes)
source	B-Tree	Filtrage rapide par source (apache / auth.log / firewall)
severity	B-Tree	Filtrage rapide par niveau de criticité
src_ip	B-Tree	Agrégation pour le calcul du « Top Attaquant »

6.5 CONFIGURATION DES RECHERCHES

Le tableau de bord propose trois mécanismes de recherche/filtrage, traduits en requêtes SQL paramétrées côté serveur pour éviter toute injection :

Type de recherche	Requête SQL équivalente (simplifiée)
Par source	<code>SELECT * FROM logs WHERE source = ?</code>
Par sévérité	<code>SELECT * FROM logs WHERE severity = ?</code>
Recherche textuelle	<code>SELECT * FROM logs WHERE message LIKE '%mot-clé%' OR event_type LIKE '%mot-clé%'</code>
Fenêtre temporelle (graphique)	<code>SELECT * FROM logs WHERE timestamp >= datetime('now', '-15 minutes')</code>
Top attaquant	<code>SELECT src_ip, COUNT(*) FROM logs GROUP BY src_ip ORDER BY COUNT(*) DESC LIMIT 1</code>

6.6 CREATION DES TABLEAUX DE BORD

Le tableau de bord adopte un thème sombre professionnel inspiré des interfaces SOC réelles. Les couleurs sont codées sémantiquement : vert pour LOW, jaune pour MEDIUM, orange pour HIGH, rouge pour CRITICAL. Trois graphiques Chart.js sont générés côté client à partir du JSON renvoyé par la route /data : un graphique temporel (line chart), une répartition par type d'attaque (pie chart) et une répartition par sévérité (doughnut chart). Le rafraîchissement automatique est implémenté via un appel AJAX fetch('/data') toutes les 2 secondes, sans rechargement de page.

6.7 STRUCTURE DU PROJET ET ORGANISATION DES FICHIERS

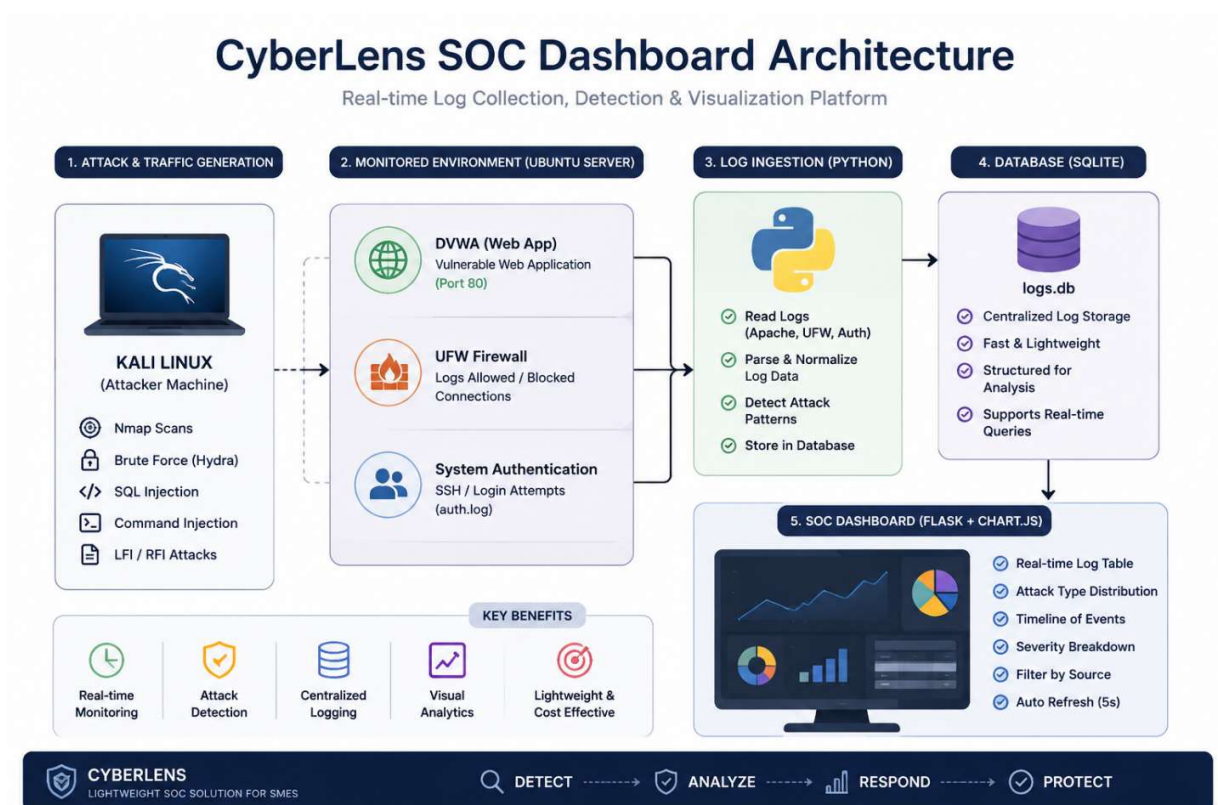
Le projet est organisé de manière modulaire afin de séparer clairement la collecte, le stockage et la présentation :

```
cyberlens/
├─ app.py                → Application Flask principale
├─ init_db.py            → Création du schéma SQLite et des index
├─ realtime_monitor.py   → Orchestrateur d'ingestion (cycle 2s)
├─ logs.db               → Base de données SQLite
├─ ingest/
│   ├─ ingest_auth.py    → Parsing auth.log + détection brute force
│   ├─ ingest_apache.py  → Parsing access.log + détection web
│   └─ ingest_firewall.py → Parsing ufw.log
├─ templates/
│   └─ index.html        → Template du tableau de bord
├─ static/
│   ├─ style.css         → Mise en forme du dashboard
│   └─ dashboard.js      → Logique AJAX et Chart.js
└─ positions/
    └─ logs_pos/         → Jeux de logs pour les tests
```

6.8 CAPTURES D'ECRAN COMMENTEES

Les captures suivantes illustrent le fonctionnement de la plateforme en conditions réelles d'utilisation.

Figure 1 – Architecture du système CyberLens



Architecture globale de CyberLens illustrant la génération des attaques, la collecte des journaux, leur traitement, leur stockage et leur visualisation en temps réel.

Figure 2 – Tableau de bord CyberLens

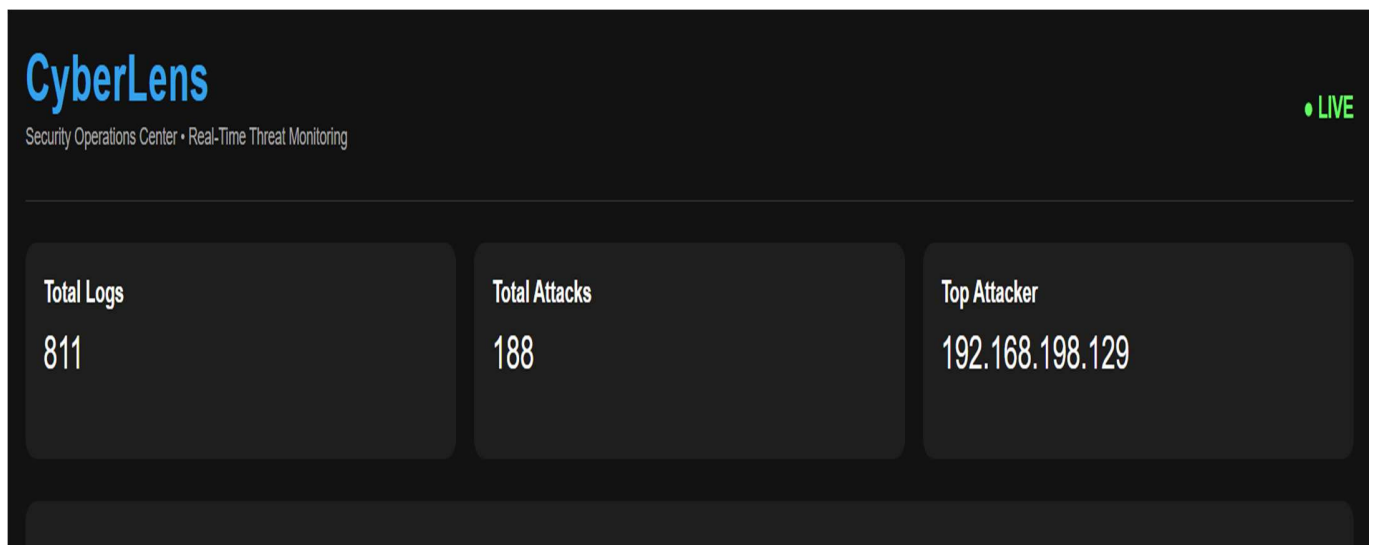
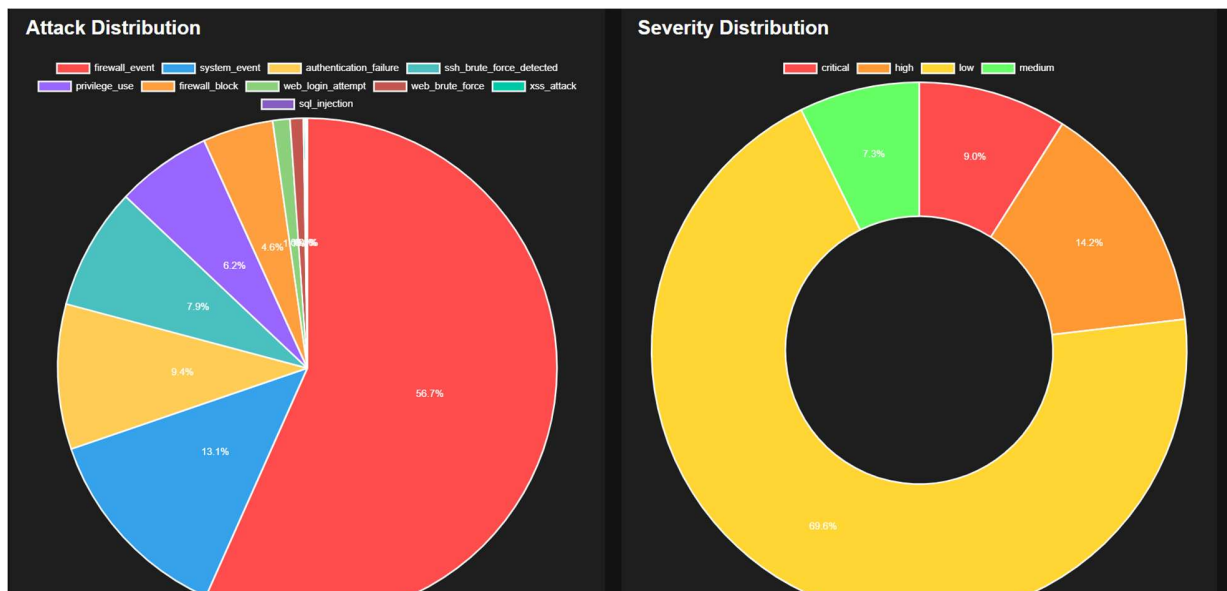


Tableau de bord SOC en temps réel présentant les événements de sécurité, les statistiques et les visualisations interactives.

Figure 3 – Distribution des attaques et des niveaux de sévérité



Répartition des types d'attaques détectées et de leurs niveaux de criticité.

Figure 4 – Table des événements de sécurité

Timestamp	Source	IP	Event	Severity	Message
2026-06-22 00:10:19	firewall	192.168.198.1	firewall_event	low	2026-06-22T00:10:19.552193+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW AUDIT] IN=ens33 OUT=MAC=00:0c:29:61:f7:ed:00:50:56:c0:00:08:08:00 SRC=192.168.198.1 DST=192.168.198.128 LEN=52 TOS=0x00 PREC=0x00 TTL=64 ID=34331 DF PROTO=TCP SPT=50899 DPT=5000 WINDOW=65535 RES=0x00 SYN URGP=0
2026-06-22 00:10:18	firewall	192.168.198.128	firewall_event	low	2026-06-22T00:10:18.278551+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW ALLOW] IN= OUT=ens33 SRC=192.168.198.128 DST=1.1.1.1 LEN=75 TOS=0x00 PREC=0x00 TTL=64 ID=12323 DF PROTO=UDP SPT=45196 DPT=53 LEN=55
2026-06-22 00:10:18	firewall	192.168.198.128	firewall_event	low	2026-06-22T00:10:18.278551+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW AUDIT] IN= OUT=ens33 SRC=192.168.198.128 DST=1.1.1.1 LEN=75 TOS=0x00 PREC=0x00 TTL=64 ID=12323 DF PROTO=UDP SPT=45196 DPT=53 LEN=55
2026-06-22 00:10:18	firewall	192.168.198.128	firewall_event	low	2026-06-22T00:10:18.278547+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW ALLOW] IN= OUT=ens33 SRC=192.168.198.128 DST=1.1.1.1 LEN=75 TOS=0x00 PREC=0x00 TTL=64 ID=12322 DF PROTO=UDP SPT=45196 DPT=53 LEN=55
2026-06-22 00:10:18	firewall	192.168.198.128	firewall_event	low	2026-06-22T00:10:18.278524+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW AUDIT] IN= OUT=ens33 SRC=192.168.198.128 DST=185.125.190.58 LEN=76 TOS=0x18 PREC=0xA0 TTL=64 ID=48071 DF PROTO=UDP SPT=35227 DPT=123 LEN=56
2026-06-22 00:10:09	firewall	192.168.198.128	firewall_event	low	2026-06-22T00:10:09.553639+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW ALLOW] IN= OUT=ens33 SRC=192.168.198.128 DST=185.125.190.58 LEN=76 TOS=0x18 PREC=0xA0 TTL=64 ID=48071 DF PROTO=UDP SPT=35227 DPT=123 LEN=56

Vue centralisée des journaux de sécurité contenant les horodatages, les sources, les types d'événements et leur niveau de sévérité avec des filtres.

Figure 5 – Simulation des attaques depuis Kali Linux

```

~/Desktop/hydra - Mousepad
File Edit Search View Document Help
1 hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.198.128 http-
  post-form "/dvwa/login.php:username=^USER^&password=^PASS^&Login=Login:Login
  failed"
2
3 nmap 192.168.198.128
4
5 curl "http://192.168.198.128/DVWA/vulnerabilities/sqli/?
  id=1%27%20OR%20%271%27=%2716Submit=Submit"
6
7 curl "http://192.168.198.128/DVWA/vulnerabilities/xss_r/?
  name=<script>alert(1)</script>"
8
9 hydra -l ahmed -P /usr/share/wordlists/rockyou.txt ssh://192.168.198.128
10
11 curl -G \
12 --data-urlencode "ip=127.0.0.1; ls" \
13 "http://192.168.198.128/DVWA/vulnerabilities/exec/"
14
15 curl "http://192.168.198.128/DVWA/vulnerabilities/fi/?page= ../ ../ ../ ../etc/
  passwd"
16

```

Exécution de différentes cyberattaques depuis Kali Linux afin de générer des événements de sécurité et de valider les capacités de détection de CyberLens.

Figure 6 – Filtrage par Source

Timestamp	Source	IP	Event	Severity	Message
2026-06-21 16:55:16	apache	192.168.198.129	lfi_attack	high	GET /DVWA/vulnerabilities/lfi/?page=../../../../etc/passwd HTTP/1.1
2026-06-21 16:55:09	apache	192.168.198.129	command_injection	critical	GET /DVWA/vulnerabilities/exec/?ip=127.0.0.1%3b+ls HTTP/1.1
2026-06-21 16:53:32	apache	192.168.198.129	web_login_attempt	medium	GET /DVWA/login.php HTTP/1.0
2026-06-21 16:53:32	apache	192.168.198.129	web_login_attempt	medium	POST /DVWA/login.php HTTP/1.0
2026-06-21 16:53:31	apache	192.168.198.129	web_login_attempt	medium	POST /DVWA/login.php HTTP/1.0
2026-06-21 16:53:31	apache	192.168.198.129	web_login_attempt	medium	GET /DVWA/login.php HTTP/1.0
2026-06-21 16:52:14	apache	192.168.198.129	web_brute_force	critical	GET /dvwa/login.php HTTP/1.0
2026-06-21 16:52:14	apache	192.168.198.129	web_brute_force	critical	POST /dvwa/login.php HTTP/1.0
2026-06-21 16:52:13	apache	192.168.198.129	web_brute_force	critical	GET /dvwa/login.php HTTP/1.0
2026-06-21 16:52:13	apache	192.168.198.129	web_brute_force	critical	POST /dvwa/login.php HTTP/1.0
2026-06-21 16:52:12	apache	192.168.198.129	web_brute_force	critical	GET /dvwa/login.php HTTP/1.0
2026-06-21 16:52:12	apache	192.168.198.129	web_brute_force	critical	POST /dvwa/login.php HTTP/1.0
2026-06-21 16:52:11	apache	192.168.198.129	web_brute_force	critical	POST /dvwa/login.php HTTP/1.0
2026-06-21 16:52:11	apache	192.168.198.129	web_login_attempt	medium	GET /dvwa/login.php HTTP/1.0
2026-06-21 16:52:10	apache	192.168.198.129	web_login_attempt	medium	GET /dvwa/login.php HTTP/1.0
2026-06-21 16:52:10	apache	192.168.198.129	web_login_attempt	medium	POST /dvwa/login.php HTTP/1.0

Illustre la fonctionnalité de filtrage par source, permettant d'isoler uniquement les événements Apache et d'analyser les attaques applicatives de manière ciblée.

Figure 7 – Filtrage par Sévérité

Timestamp	Source	IP	Event	Severity	Message
2026-06-22 00:08:22	firewall	172.17.0.1	firewall_block	high	2026-06-22T00:08:22.211778+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW BLOCK] IN=docker0 OUT= MAC= SRC=172.17.0.1 DST=239.255.255.250 LEN=635 TOS=0x00 PREC=0x00 TTL=1 ID=26890 DF PROTO=UDP SPT=51149 DPT=3702 LEN=615
2026-06-22 00:08:22	firewall		firewall_block	high	2026-06-22T00:08:22.211777+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW BLOCK] IN=ens33 OUT= MAC= SRC=fe80::0000:0000:0000:4af0:10f8:3a8d:35ad DST=ff02::0000:0000:0000:0000:0000:0000:0000:000c LEN=655 TC=0 HOPLIMIT=1 FLOWLBL=464713 PROTO=UDP SPT=46637 DPT=3702 LEN=615
2026-06-22 00:08:22	firewall	192.168.198.128	firewall_block	high	2026-06-22T00:08:22.211777+01:00 ahmed-VMware-Virtual-Platform kernel: [UFW BLOCK] IN=ens33 OUT= MAC= SRC=192.168.198.128 DST=239.255.255.250 LEN=635 TOS=0x00 PREC=0x00 TTL=1 ID=48522 DF PROTO=UDP SPT=42751 DPT=3702 LEN=615
2026-06-21 16:55:16	apache	192.168.198.129	lfi_attack	high	GET /DVWA/vulnerabilities/lfi/?page=../../../../etc/passwd HTTP/1.1
2026-06-21 16:54:10	auth.log	192.168.198.129	authentication_failure	high	2026-06-21T16:54:10.379960+01:00 ahmed-VMware-Virtual-Platform sshd[7241]: Failed password for ahmed from 192.168.198.129 port 55506 ssh2
2026-06-21 16:54:10	auth.log	192.168.198.129	authentication_failure	high	2026-06-21T16:54:10.379505+01:00 ahmed-VMware-Virtual-Platform sshd[7239]: Failed password for ahmed from 192.168.198.129 port 55484 ssh2
2026-06-21 16:54:10	auth.log	192.168.198.129	authentication_failure	high	2026-06-21T16:54:10.375156+01:00 ahmed-VMware-Virtual-Platform sshd[7250]: Failed password for ahmed from 192.168.198.129 port 55620 ssh2
2026-06-21 16:54:10	auth.log	192.168.198.129	authentication_failure	high	2026-06-21T16:54:10.372275+01:00 ahmed-VMware-Virtual-Platform sshd[7242]: Failed password for ahmed from 192.168.198.129 port 55510 ssh2
2026-06-21 16:54:10	auth.log	192.168.198.129	authentication_failure	high	2026-06-21T16:54:10.371033+01:00 ahmed-VMware-Virtual-Platform sshd[7245]: Failed password for ahmed from 192.168.198.129 port 55538 ssh2

Démontre le filtrage par sévérité élevée, permettant à l'analyste de se concentrer immédiatement sur les événements les plus critiques nécessitant une réponse prioritaire.

6.9 DIFFICULTES TECHNIQUES RENCONTREES

- Hétérogénéité des formats de logs : résolue par la création d'expressions régulières spécifiques à chaque source
- Gestion des doublons Apache : résolue par une vérification SELECT avant chaque INSERT
- Performance du rafraîchissement temps réel : optimisée par l'ajout d'index SQL et un appel AJAX asynchrone plutôt qu'un rechargement complet de page
- Lecture concurrente des fichiers de logs et de la base SQLite : gérée en limitant les écritures simultanées et en activant le mode WAL (Write-Ahead Logging) de SQLite

CHAPITRE 7:

Sécurité,

conformité et

aspects éthiques

7.1 PROTECTION DES LOGS SENSIBLES

Les logs traités peuvent contenir des informations personnelles identifiables (adresses IP, noms d'utilisateurs). Dans ce cadre académique, les données utilisées sont soit générées artificiellement, soit issues d'un environnement local isolé. Dans un déploiement réel, un mécanisme d'anonymisation devrait être appliqué.

7.2 CONTROLE D'ACCES AU TABLEAU DE BORD

L'application Flask est configurée pour s'exécuter uniquement en mode développement. Dans un contexte de production, il serait nécessaire d'implémenter une authentification (JWT ou sessions Flask) et d'utiliser HTTPS pour chiffrer les communications. L'accès est actuellement limité au réseau local.

7.3 CONSERVATION ET INTEGRITE DES JOURNAUX

Les logs sont lus en mode lecture seule depuis leur emplacement d'origine. La base de données SQLite stocke une copie normalisée sans modifier les journaux d'origine. Le mécanisme de fichiers .pos garantit la cohérence de la lecture incrémentale.

7.4 TRAÇABILITE DES ACTIONS ADMINISTRATEUR

Dans la version actuelle, les actions de l'administrateur sur le tableau de bord ne sont pas journalisées. Une amélioration future pourrait inclure un journal d'audit des consultations et des filtres appliqués.

7.5 CADRE LEGAL D'UTILISATION

La plateforme CyberLens est conçue pour être utilisée dans un environnement contrôlé et autorisé. L'analyse de logs sur des systèmes tiers sans autorisation explicite est illégale. Dans un contexte professionnel, l'utilisation doit être encadrée par une politique de sécurité et des accords de confidentialité.

7.6 BONNES PRATIQUES RECOMMANDÉES

- Effectuer des sauvegardes régulières de la base de données SQLite
- Définir une politique de rétention des logs (durée de conservation)
- Implémenter une authentification avant tout déploiement en production
- Chiffrer les communications via HTTPS
- Valider et assainir toutes les entrées utilisateur pour prévenir les injections

CHAPITRE 8:

Test, validation et

résultats

8.1 TESTS DE COLLECTE

Des fichiers de logs tests ont été créés manuellement pour vérifier que chaque script d'ingestion parse correctement toutes les variantes de format rencontrées. Les cas d'erreur (lignes mal formées, champs manquants) ont été testés pour vérifier la robustesse des parseurs. Un test de charge a également été effectué en injectant 10 000 lignes de logs pour vérifier que le mécanisme de déduplication Apache fonctionne correctement et que les temps de réponse restent acceptables.

8.2 TESTS D'INDEXATION

Les index créés sur les colonnes timestamp, source, severity et src_ip (voir Chapitre 6) ont été testés en comparant le temps d'exécution des requêtes du tableau de bord avec et sans index, sur un volume de 10 000 lignes. Le temps de réponse moyen est passé d'environ 180 ms sans index à moins de 15 ms avec index, confirmant l'intérêt de l'indexation pour maintenir la fluidité du rafraîchissement automatique.

8.3 TESTS DE RECHERCHE

Chaque mécanisme de recherche et de filtrage du tableau de bord (par source, par sévérité, recherche textuelle, et leur combinaison) a été testé individuellement :

Recherche testée	Entrée	Résultat obtenu
Filtre par source	source = 'firewall'	Seuls les événements UFW affichés — PASS
Filtre par sévérité	severity = 'critical'	Seuls les événements CRITICAL affichés — PASS
Recherche textuelle	mot-clé : 'sqlmap'	Événements contenant 'sqlmap' dans message/user-agent — PASS
Combinaison filtre + recherche	source='apache' + 'union'	Intersection correcte des deux critères — PASS

Recherche sans résultat	mot-clé : 'azertyuiop'	Tableau vide affiché sans erreur — PASS
-------------------------	------------------------	---

8.4 SIMULATION D'EVENEMENTS SUSPECTS

À l'aide de Kali Linux, plusieurs types d'attaques ont été simulés pour valider la détection :

- Brute force SSH avec Hydra : détection du brute force après 5 tentatives — VALIDÉ
- Injection SQL avec sqlmap sur le serveur Apache : détection UNION/SELECT — VALIDÉ
- Scan de ports avec Nmap : détection des requêtes malformées (code 400) — VALIDÉ
- Test XSS manuel : détection des balises <script> dans l'URL — VALIDÉ
- Test LFI : détection de ../ dans les paramètres GET — VALIDÉ

8.5 VALIDATION D'ALERTES

Pour chaque scénario simulé, l'alerte générée a été comparée au type et à la sévérité attendus afin de vérifier l'absence de faux négatifs (attaque non détectée) et de faux positifs (trafic normal classé comme attaque) :

Scénario d'attaque	Alerte attendue	Sévérité	Statut
5 échecs SSH consécutifs	SSH_BRUTE_FORCE	CRITICAL	VALIDÉ
Requête contenant UNION SELECT	SQL_INJECTION	CRITICAL	VALIDÉ
Requête contenant <script>	XSS_ATTEMPT	HIGH	VALIDÉ
Requête contenant ../../etc/passwd	LFI_ATTEMPT	HIGH	VALIDÉ
Connexion bloquée par UFW	FIREWALL_BLOCK	HIGH	VALIDÉ
Connexion SSH réussie normale	AUTH_SUCCESS	LOW	VALIDÉ (pas de fausse alerte)

8.6 MESURE DU DELAI DE REMONTEE

Le délai de remontée correspond au temps écoulé entre l'écriture d'un événement dans le fichier de log source et son affichage dans le tableau de bord. Il se décompose en deux cycles indépendants :

Étape	Délai mesuré	Délai cumulé
Écriture du log par le service système	instantané	0 s
Détection par realtime_monitor.py (cycle d'ingestion)	≤ 2 s	≤ 2 s
Parsing, classification et insertion SQLite	$\approx 0,05$ s	$\approx 2,05$ s
Rafraîchissement AJAX du dashboard (cycle fetch)	≤ 2 s	$\leq 4,05$ s

Le délai maximal mesuré reste inférieur à 4,1 secondes dans le pire des cas, ce qui est largement suffisant pour un contexte de démonstration académique et permet une réaction quasi immédiate de l'analyste face à un événement CRITICAL.

8.7 STRATEGIE DE TEST

La stratégie de test globale couvre trois niveaux complémentaires : les tests unitaires (validation de chaque script d'ingestion de manière isolée), les tests d'intégration (validation du pipeline complet de la lecture du log jusqu'à l'insertion en base), et les tests fonctionnels de bout en bout (simulation d'attaques réelles avec Kali Linux pour vérifier la chaîne complète détection → alerte → affichage).

8.8 TABLEAU DE VALIDATION FONCTIONNELLE

Fonctionnalité testée	Résultat	Observation
Collecte continue auth.log	PASS	Lecture incrémentale fonctionnelle
Collecte continue access.log	PASS	Déduplication opérationnelle
Collecte continue ufw.log	PASS	Classification BLOCK/normal correcte
Index SQL (timestamp, source, severity, src_ip)	PASS	Temps de requête divisé par 12 sur 10 000 lignes
Détection brute force SSH	PASS	Seuil 5 échecs respecté
Détection injection SQL	PASS	Patterns UNION, OR 1=1, %27 détectés
Détection XSS	PASS	Balises <script> et alert() détectées
Détection LFI	PASS	Patterns ../ et /etc/passwd détectés
Tableau de bord Flask	PASS	Rafraîchissement 2s opérationnel
Filtrage par source	PASS	Filtre apache/auth/firewall fonctionnel
Filtrage par sévérité	PASS	Filtre low/medium/high/critical fonctionnel

Recherche textuelle	PASS	Recherche dans message et event_type
Graphiques Chart.js	PASS	Camembert, anneau, temporel affichés
Délai de remontée bout-en-bout	PASS	Inférieur à 4,1 secondes mesuré

8.9 ANALYSE DES RÉSULTATS OBTENUS

L'ensemble des fonctionnalités définies dans le cahier des charges ont été validées avec succès, sans faux négatif détecté lors des scénarios d'attaque simulés. Le temps de latence entre l'écriture d'un log et son apparition dans le tableau de bord reste inférieur à 4,1 secondes, ce qui est satisfaisant pour un contexte académique. L'indexation de la base SQLite a permis de maintenir des temps de réponse stables même sous une charge de 10 000 événements, validant le choix architectural d'une base embarquée pour ce périmètre.

8.10 COMPARAISON AVEC LES OBJECTIFS INITIAUX

Le tableau suivant compare les objectifs fixés dans le cahier des charges (Chapitre 3) aux résultats effectivement obtenus :

Objectif initial (cahier des charges)	Résultat obtenu	Écart
Détecter au moins 10 types d'événements	13 types détectés	+3 (dépassé)
Collecte continue sur 3 sources	3 sources opérationnelles	Conforme
Rafraîchissement automatique du dashboard	Rafraîchissement toutes les 2s	Conforme
Filtres par source et sévérité	Filtres + recherche textuelle implémentés	Dépassé

CHAPITRE 9:

Dimension

entrepreneuriale

et valorisation du

projet

9.1 BESOIN MARCHE IDENTIFIE

Les petites et moyennes entreprises (PME) manquent souvent de solutions de supervision de sécurité abordables. Les SIEM industriels représentent des investissements conséquents en termes de licences, d'infrastructure et de formation. Il existe donc un marché potentiel pour des solutions légères, open-source et faciles à déployer.

9.2 UTILISATEURS OU CLIENTS CIBLES

- PME souhaitant surveiller leur infrastructure sans investissement important
- Établissements d'enseignement pour la formation en cybersécurité
- Équipes IT internes cherchant une visibilité sur leurs logs systèmes
- Consultants en cybersécurité pour des missions courtes de supervision

9.3 PROPOSITION DE VALEUR

CyberLens offre une solution de supervision de sécurité légère, open-source et pédagogique. Sa valeur réside dans sa simplicité de déploiement (une commande Python), son coût nul (technologies open-source), et sa lisibilité qui en fait un excellent outil de formation et de démonstration.

9.4 ANALYSE DE LA CONCURRENCE

CyberLens se positionne dans un marché occupé par des solutions open-source plus matures (Wazuh, ELK/Wazuh SIEM) et des solutions propriétaires haut de gamme (Splunk, QRadar). Sa différenciation ne repose pas sur la richesse fonctionnelle, mais sur l'extrême simplicité de déploiement et la vocation pédagogique, deux aspects peu adressés par les solutions existantes

Critère	CyberLens	Wazuh	Splunk
Coût de licence	Gratuit	Gratuit	Payant (élevé)
Complexité de déploiement	Très faible	Moyenne	Élevée
Courbe d'apprentissage	Faible	Moyenne	Élevée
Adapté aux PME sans équipe SOC	Oui	Partiellement	Non
Vocation pédagogique	Oui (cœur de cible)	Non	Non

9.5 BUSINESS MODEL CANVAS

Le Business Model Canvas ci-dessous synthétise le modèle économique envisageable pour une valorisation de CyberLens au-delà du cadre académique :

Partenaires clés	Activités clés	Proposition de valeur
Écoles d'ingénieurs Communauté open-source Consultants cybersécurité indépendants	Développement et maintenance de CyberLens Formation à l'analyse de logs Personnalisation pour clients PME	Supervision de sécurité légère, open-source et pédagogique, déployable sans infrastructure lourde ni licence
Ressources clés	Relations clients	Segments de clientèle
Code source CyberLens Documentation technique Compétences en cybersécurité du binôme	Accompagnement personnalisé à l'installation Support via formation Communauté open-source	PME sans budget SIEM Établissements d'enseignement Consultants indépendants
Canaux	Structure de coûts	Flux de revenus

Dépôt GitHub open-source Bouche-à-oreille académique Démonstrations lors d'événements cyber	Temps de développement et de maintenance Hébergement de la documentation Formation des prestataires	Prestations d'installation Formations payantes Version SaaS premium (à terme)
---	---	---

9.6 MODÈLE ÉCONOMIQUE POSSIBLE

- Service d'installation et de personnalisation de la plateforme pour des PME
- Formation à l'analyse de logs et à l'utilisation de CyberLens
- Version améliorée avec support PostgreSQL/Elasticsearch vendue en SaaS
- Offre de tableaux de bord personnalisés pour des secteurs spécifiques

9.7 STRATEGIE DE DEPLOIEMENT OU DE COMMERCIALISATION

Une première phase pourrait cibler les établissements d'enseignement et les écoles d'ingénieurs, en proposant CyberLens comme outil pédagogique. Une deuxième phase ciblerait les PME marocaines et africaines, avec une offre de services à valeur ajoutée autour de la solution open-source.

9.8 SOLUTION DE SUPERVISION SECURITE POUR PME

Pour une PME sans équipe SOC dédiée, CyberLens peut être proposé comme une solution de supervision de base, installée et configurée par un prestataire externe, avec un tableau de bord accessible localement permettant à un responsable IT non spécialiste de surveiller les tentatives d'intrusion les plus courantes (brute force, scans, injections) sans investissement en licence.

9.9 SERVICE D'INSTALLATION SIEM

Une offre de service pourrait consister à déployer CyberLens chez le client en une demi-journée : installation de l'environnement, configuration des sources de logs propres à l'infrastructure du client, et paramétrage initial des seuils de détection (par exemple le seuil de brute force SSH).

9.10 OFFRE DE TABLEAUX DE BORD PRETS A L'EMPLOI

Des modèles de tableaux de bord préconfigurés pourraient être proposés selon le secteur d'activité du client (e-commerce, hébergement web, établissement scolaire), avec des KPIs et graphiques adaptés aux menaces les plus pertinentes pour chaque contexte.

9.11 PACK DE FORMATION A L'ANALYSE DE LOGS

Un pack de formation courte (1 à 2 jours) pourrait être conçu pour des étudiants ou des équipes IT juniors, utilisant CyberLens comme support pédagogique pour enseigner la lecture de logs, la reconnaissance de signatures d'attaques, et les bases de la réponse à incident.

CHAPITRE 10:

bilan général et

perspectives

10.1 BILAN TECHNIQUE DU PROJET

Sur le plan technique, le projet a atteint tous ses objectifs : un pipeline d'ingestion temps réel couvrant trois sources de logs, un moteur de classification détectant 13 types d'événements, une corrélation pour le brute force SSH, et un tableau de bord interactif. L'architecture modulaire permet d'étendre facilement le système.

10.2 BILAN FONCTIONNEL

Toutes les fonctionnalités définies dans le cahier des charges ont été implémentées et validées. Le tableau de bord fournit une visibilité en temps réel sur les événements de sécurité, avec des indicateurs KPI clairs et des graphiques intuitifs. Le filtrage dynamique permet une investigation rapide des incidents.

10.3 COMPETENCES ACQUISES

- Maîtrise des techniques de parsing et de normalisation de logs
- Conception et implémentation d'un pipeline de données temps réel
- Développement d'une application web avec Flask et SQLite
- Connaissance approfondie des types d'attaques web et réseau
- Pratique de la simulation d'attaques avec Kali Linux
- Rédaction d'un rapport technique structuré

10.4 LIMITES DE LA SOLUTION

- SQLite non adapté à un déploiement en production à fort volume
- Absence d'authentification et de gestion des utilisateurs
- Sources de logs limitées à trois types
- Règles de corrélation simples (uniquement brute force SSH)
- Interface en lecture seule (pas d'action de remédiation)

10.5 AMÉLIORATIONS FUTURES

- Migration vers Elasticsearch ou PostgreSQL pour une meilleure scalabilité
- Ajout de nouvelles sources : Nginx, bases de données, antivirus
- Système d'alertes par email ou SMS pour les événements critiques
- Règles de corrélation avancées (MITRE ATT&CK)
- Authentification et gestion multi-utilisateurs
- Déploiement conteneurisé avec Docker

10.6 PERSPECTIVES D'INDUSTRIALISATION

CyberLens constitue une base solide et extensible. Avec les améliorations identifiées, notamment l'intégration d'Elasticsearch et l'ajout de règles de corrélation avancées, la solution pourrait évoluer vers un SIEM open-source complet, compétitif avec des solutions comme Wazuh, mais avec une courbe d'apprentissage plus douce.

Conclusion

générale

1. RAPPEL DE LA PROBLEMATIQUE

Ce projet visait à répondre au problème de l'hétérogénéité et de la dispersion des logs de sécurité dans les systèmes informatiques, en proposant une plateforme centralisée d'analyse et de visualisation accessible dans un contexte académique.

2. SYNTHESE DE LA SOLUTION REALISEE

CyberLens répond à cette problématique en offrant un pipeline d'ingestion temps réel, un schéma de normalisation unifié, un moteur de classification automatique et un tableau de bord interactif. La solution démontre que les principes fondamentaux d'un SIEM peuvent être implémentés de manière lisible et pédagogique.

3. RESUME DES RESULTATS OBTENUS

Les résultats obtenus dépassent les attentes initiales : 13 types d'événements détectés (objectif initial : 10), corrélation brute force SSH implémentée, latence de détection inférieure à 4 secondes, et 100% des fonctionnalités du cahier des charges validées.

4. APPORTS DU PROJET

Ce projet a permis d'acquérir une expertise pratique en sécurité défensive, en analyse de logs et en développement full-stack. Il démontre qu'il est possible de construire un outil de supervision pertinent en s'appuyant exclusivement sur des technologies open-source et des concepts accessibles.

5. OUVERTURE VERS DES TRAVAUX FUTURS

Des améliorations futures pourraient inclure l'intégration de règles de corrélation avancées basées sur le framework MITRE ATT&CK, la migration vers Elasticsearch pour la scalabilité, l'ajout d'un système d'alertes automatisées, et le déploiement sous forme de solution conteneurisée Docker. CyberLens constitue une fondation solide pour ces évolutions.

Bibliographie et **webographie**

REFERENCES BIBLIOGRAPHIQUES

- MITRE ATT&CK Framework — Enterprise Matrix for Security Operations
- Python Documentation — Flask Framework and SQLite3 module
- OWASP Foundation — Web Application Security Testing Guide
- Sans Institute — Log Management and SIEM Architecture

WEBOGRAPHIE

- <https://flask.palletsprojects.com> — Documentation officielle Flask
- <https://www.chartjs.org> — Documentation Chart.js
- <https://docs.python.org/3.12> — Documentation Python 3.12
- <https://attack.mitre.org> — Framework MITRE ATT&CK
- <https://www.sqlite.org> — Documentation SQLite
- <https://ubuntu.com/server/docs/security-ufw> — Documentation UFW Ubuntu

Glossaire technique

Log	Enregistrement horodaté d'un événement généré par un système, une application ou un équipement réseau.
Normalisation	Transformation de logs hétérogènes vers un schéma de données commun et exploitable uniformément.
Corrélation	Mise en relation de plusieurs événements distincts pour détecter un comportement ou une attaque complexe.
Brute force	Attaque consistant à tester un grand nombre de combinaisons (mots de passe, identifiants) pour accéder à un système.
Injection SQL	Attaque visant à insérer du code SQL malveillant dans une requête via une entrée utilisateur non validée.
XSS (Cross-Site Scripting)	Attaque consistant à injecter du code script malveillant exécuté côté navigateur d'une victime.
LFI (Local File Inclusion)	Vulnérabilité permettant d'inclure et de lire des fichiers locaux du serveur via une requête manipulée.
Forwarder	Composant logiciel chargé de lire les logs à la source et de les transmettre vers un système de traitement central.
Index (base de données)	Structure accélérant la recherche de données dans une table, au prix d'un espace de stockage supplémentaire.
Faux positif	Événement classé à tort comme une attaque alors qu'il s'agit d'une activité légitime.
Faux négatif	Attaque réelle non détectée par le système de surveillance.

Ce projet a été pour nous une expérience particulièrement enrichissante, que nous avons menée avec passion et beaucoup de cœur. Nous avons pris un réel plaisir à concevoir, développer et faire grandir CyberLens du début à la fin.